

1. Dictionary and hashing

Dictionary ADT

Dictionary is a key-value map, and contain several operations such as:

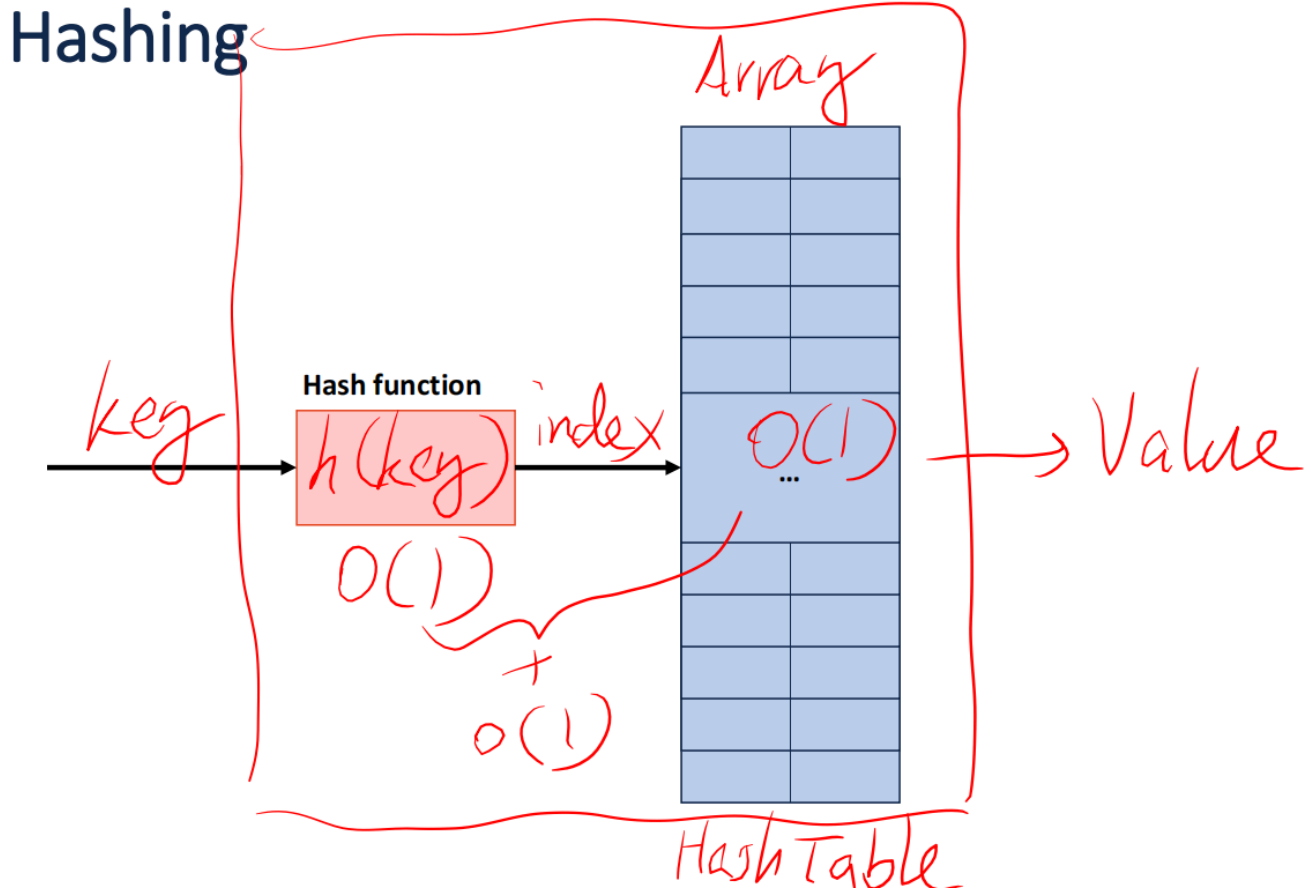
- insert(k, v)
- remove(k)
- find(k)

Dictionary.h

```
1  #ifndef DICTIONARY_H
2  #define DICTIONARY_H
3
4  template <class K, class V>
5  class Dictionary {
6      public:
7          void insert(K & k, V & v);
8          void remove(const K & k);
9          V & find(const K & k) const;
10         associative array []
11         V & operator[] (const K & k) const;
12         d["wade"]
13
14
15     private:
16 };
17
18
19 #endif
20
21
22
```

2. Hashing

2.1 structure



a hash table contains three things:

- a hash function
- an array
- collision resolution

perfect hash function = injective = no collision

a hash function contains two things:

- hash: hash code= $h(\text{key})$
this may be a very large number
- compression: index= $c(\text{hash code})$
to compress the hash code into $0, 1, 2, \dots, m-1$, if the has table has only m elements

2.2 good hash function

Hash Function

Characteristics of a good hash function:

1. Computation Time:

Runs in constant time, $O(1)$

2. Deterministic:

No random element
if $k_1 = k_2 \rightarrow h(k_1) = h(k_2)$

3. Satisfy the SUHA:

Simple Uniform hashing assumption

For keys k, j
where $k \neq j$

$$P(h(k) = h(j)) = 1/N$$

2.3 collision handling - separate chaining / open hashing

(Example of open hashing)

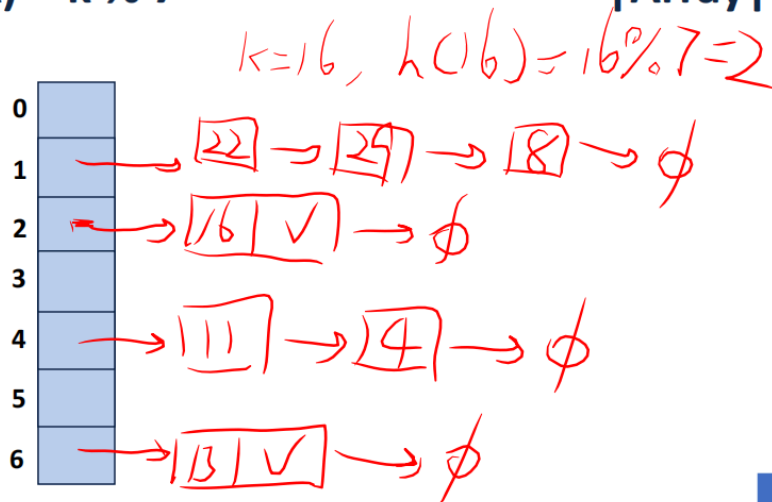
Collision Handling: Separate Chaining

$S = \{ 16, 8, 4, 13, 29, 11, 22 \}$

$|S| = n$

$h(k) = k \% 7$

$|Array| = N$



	Worst Case	SUHA
Insert	$O(1)$	
Remove/Find	$O(n) \equiv O(n/N)$	

- Each slot in the table holds not just a single element, but a container.
- A common implementation is a linked list, though other structures can also be used.

	Worst Case	SUHA
Insert	$O(1)$	
Remove/Find	$O(n)$	$\approx O(n/N)$

insert:

1. first compute hash, to find the bucket
2. insert the new element directly at the head of the linked list

remove:

1. first locate the bucket
 2. search for the target element in the chain associated with this bucket
- So in worst case, all elements are hashed into the same bucket.

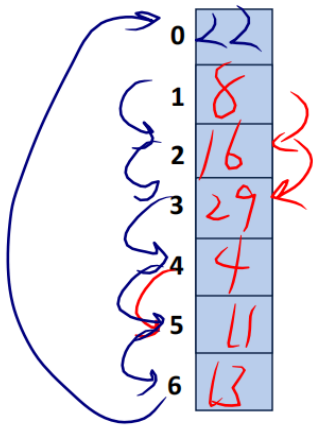
And in SUHA case, the length of chain is n/m , therefore complexity $O(n/m) = O(\alpha)$.

2.4 collision handling - Probe-Based Hashing

(Example of closed hashing)

Collision Handling: Probe-based Hashing

$S = \{ 16, 8, 4, 13, 29, 11, 22 \}$ $|S| = n$
 $h(k) = k \% 7$ $|Array| = N$



Try $h(k) = (k + 0) \% 7$, if full...

Try $h(k) = (k + 1) \% 7$, if full...

Try $h(k) = (k + 2) \% 7$, if full...

Try ... $2 \leq 1.0$ $\frac{n}{N} = \frac{7}{7}$

2.4.1 Key concepts

- After a collision, the pointer does not wrap around to the next position but continues searching for the next slot within the array
- This method is also known as:
 - closed hashing
 - open addressing

2.4.2 method

- First, calculate the initial position $h(k)$.
- If the slot is occupied, continue trying using a probe sequence:

$h(k, 0), h(k, 1), h(k, 2), \dots$

- Continue until an empty slot is found, or the target key is found during the search, or it is determined that the key does not exist.

2.4.3 Restriction to α

$$\alpha = n / N < 1$$

2.4.4 Linear Probing

$$h(k, i) = (h(k) + i) \bmod N$$

Problem: Primary Clustering

2.4.5

$$h(k, i) = (h_1(k) + i * h_2(k)) \bmod N$$

2.5 Running time

Running Times

The expected number of probes for find(key) under SUHA

Linear Probing:

- Successful: $\frac{1}{2}(1 + 1/(1-\alpha))$
- Unsuccessful: $\frac{1}{2}(1 + 1/(1-\alpha))^2$

(Don't memorize these equations, no need.)

Double Hashing:

- Successful: $1/\alpha * \ln(1/(1-\alpha))$
- Unsuccessful: $1/(1-\alpha)$

Instead, observe:

- As α increases:

We have slower hash table operations

- If α is constant:

Running time does not change

Separate Chaining:

- Successful: $1 + \alpha/2$
- Unsuccessful: $1 + \alpha$

3. BTree

3.1 BTree analysis

1. Key concepts

- B-Tree m -way tree
- There can be several keys in a certain node

2. BTree Properties

Btree Properties

A **BTrees** of order **m** is an m -way tree:

- All keys within a node are ordered
- All nodes contain no more than **$m-1$** keys.
- All internal nodes have exactly **one more child than keys**
- Root nodes can be a leaf or have **$[2, m]$** children.
- All non-root, internal nodes have **$[\text{ceil}(m/2), m]$** children.
- All leaves are on the same level

3. height

- minimum: $2t^{h-1}$
- maximum: $m^{(h+1)}-1$

3.2 BTree operation

3.2.1 search

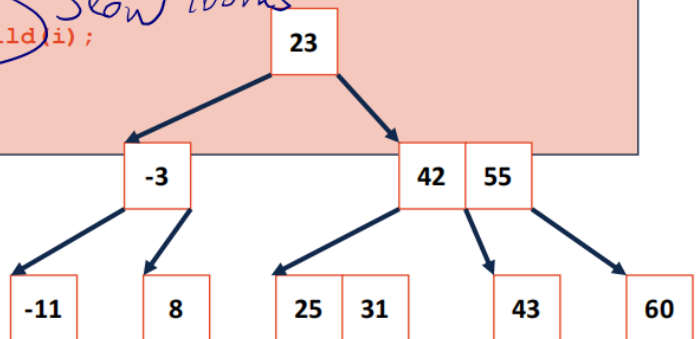
BTree Search

```
1 bool Btree::_exists(BTreeNode & node, const K & key) {  
2     // Goes through a node, find location  
3     unsigned i;  
4     for ( i = 0; i < node.keys_ct_ && key < node.keys_[i]; i++) {}  
5  
6     if ( i < node.keys_ct_ && key == node.keys_[i] ) {  
7         return true;  
8     }  
9  
10    if ( node.isLeaf() ) { } Base case  
11        return false;  
12    } else {  
13        BTreeNode nextChild = node.fetchChild(i);  
14        return _exists(nextChild, key);  
15    }  
16 }
```

Can do BSearch

Slow loops

Recursion



3.2.2 others

check lab_btree